

System Design — Complete Beginner-Friendly Notes

Detailed textbook-style edition — preserving the complete conversation content.

System Design — Complete Beginner-Friendly Notes

System design is one of the most important topics for your Google SWE preparation. The goal is **not** to memorize architectures. The goal is to learn how to take a real-world problem and systematically turn it into a reliable software system.

I'll build the topic from **zero technical knowledge**, then gradually connect it to the kind of systems you will eventually design in Google interviews.

1. What Is System Design?

Let's start without computers.

Imagine you are asked:

"Build a food-delivery service for 10 million customers."

Before writing any code, you need to answer questions like:

- Where is customer information stored?
- Where are restaurant details stored?
- How does a customer place an order?
- How does the restaurant receive the order?
- How does the delivery person receive it?
- What happens if 100,000 people order at the same time?
- What happens if a server crashes?
- How do we prevent duplicate orders?
- How do we make the application fast?
- How do we keep it available when something fails?

System design is the process of answering these questions and designing the overall software system that solves the problem.

In simple terms:

System design is deciding how different software components work together to provide a reliable, scalable application.

2. Think of System Design Like Designing a City

This analogy will help you understand almost everything.

Imagine designing a city.

You need:

- houses
- roads
- traffic signals
- water supply
- electricity
- hospitals
- schools
- garbage collection
- emergency services
- communication systems

You don't start by deciding what color the buildings should be.

You first determine:

How should the entire city function?

Software systems are similar.

A large application may contain:

```text

Users

↓

Internet

↓

Load Balancer

↓

Application Servers

↓

Cache

↓

Database



Storage



Message Queue



Background Workers

Each component has a purpose.

---

### # 3. Why Do We Need System Design?

A tiny application can often run on one computer.

For example:

```
```text
User
↓
Python application
↓
Database
```

This might be completely sufficient for a small internal application.

But imagine:

```
```text
1 user
becomes:
```text
10 million users
```

Now problems appear.

The application may become:

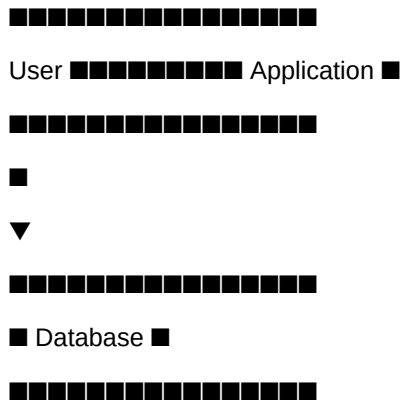
- slow
- unavailable
- overloaded
- expensive
- difficult to maintain
- vulnerable to failures

System design helps us solve these problems systematically.

4. Small System vs Large System

Small application

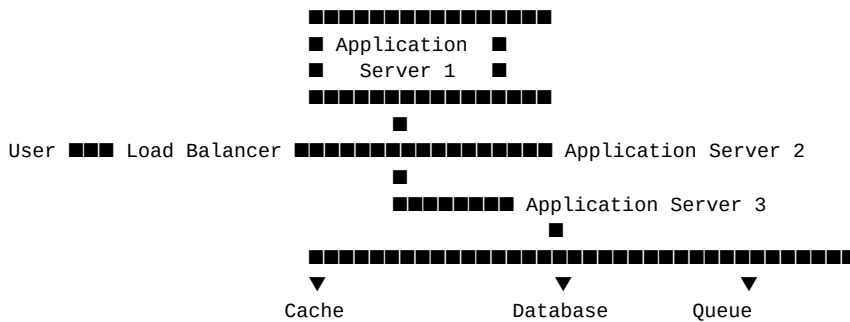
```
```text
```



One application server may be enough.

### ### Large application

```
```text
```



Now architecture matters significantly.

— — —

5. System Design Is Not Just Drawing Boxes

A common beginner mistake is thinking:

"System design means drawing architecture diagrams."

No.

The diagram is only the **visual representation** of your decisions.

The actual work is understanding:

```
```text
```

## Requirements

↓

Scale



APIs



Data



Architecture



Performance



Reliability



Security



Monitoring



Cost

---

## # 6. The Two Major Types of System Design

You will frequently hear:

### ## 6.1 High-Level Design – HLD

High-Level Design focuses on the **overall architecture**.

For example:

```
```text
Client
  ↓
Load Balancer
  ↓
Application Servers
  ↓
Cache
  ↓
Database
```

Questions include:

- What components exist?
- How do they communicate?
- What database should we use?
- Do we need caching?
- Do we need queues?
- How do we scale?

7. Low-Level Design — LLD

Low-Level Design focuses more on the internal implementation.

For example:

```
```python
```

```
class User:
```

```
def create_order():
```

```
...
```

Questions include:

- What classes exist?
- What interfaces exist?
- What methods exist?
- What design patterns are useful?
- How should objects interact?

For Google SWE preparation, **HLD/system design and coding/DSA are particularly important**, while LLD knowledge supports

---

### # 8. The Fundamental System Design Vocabulary

Before designing systems, you need to understand several words.

We'll build these slowly.

---

### # 9. Client

A **client** is something that requests a service.

Examples:

- browser
- mobile application
- desktop application
- another backend service

For example:

```
```text
Chrome Browser
  ↓
GET /users/123
```

The browser is the client.

10. Server

A server is a computer/program that receives requests and provides responses.

Example:

```
```text
```

Client

↓

"Give me user 123"

↓

Server

↓

"Here is user 123"

A server may:

- execute business logic
- access databases
- call other services
- process files
- authenticate users

---

#### # 11. Request

A request is a message asking a system to perform an operation.

For example:

```
```text
```

POST /documents

with:

```
```json
```

```
{
 "name": "Engine Requirements",
 "type": "requirement"
}
```

---

#### # 12. Response

The server sends a response.

For example:

```
```json
```

```
{  
  "id": "doc-123",  
  "name": "Engine Requirements",  
  "type": "requirement"  
}
```

13. API

API stands for:

Application Programming Interface

Don't let the name scare you.

Think of an API as a **menu at a restaurant**.

The customer doesn't need to know how the kitchen works.

The menu tells them:

```
```text
```

GET /documents

POST /documents

GET /documents/{id}

DELETE /documents/{id}

The customer chooses an operation.

The kitchen handles the implementation.

Similarly:

```
```text
```

Client

↓

API

↓

Backend

The API is the agreed interface between them.

14. HTTP

HTTP is one of the primary protocols used for communication between clients and web servers.

Common HTTP methods:

```
```text
```

GET

POST

PUT

PATCH

DELETE

```
GET
```

Retrieve something.

```
```text
```

GET /users/123

POST

Create something.

```text

POST /users

### PUT

Replace/update something.

```text

PUT /users/123

PATCH

Partially update something.

```text

PATCH /users/123

### DELETE

Delete something.

```text

DELETE /users/123

15. Database

A database stores information.

For example:

```text

Users

Orders

Documents

Products

Payments

A relational database might look like:

```text

Users

id | name | email

1 | John | john@x.com

2 | Jane | jane@x.com

Examples include:

- PostgreSQL
- MySQL
- SQL Server

16. Why Not Just Store Everything in Memory?

You already encountered this in your Project 1.

An in-memory dictionary might look like:

```
```python
documents = {}

It's fast.

But what happens if the application crashes?

The data disappears.

Therefore:

```text
Application memory
```

is not generally sufficient as durable storage.

A database provides persistent storage.

Your Project 1 progression is therefore:

```
```text
In-memory repository
↓
PostgreSQL repository
```

That is already a small system-design decision.

---

# 17. Cache

A cache stores frequently accessed information temporarily so it can be retrieved faster.

Imagine a library.

If you frequently ask for the same book, the librarian might keep it on the desk instead of going to the storage room e

That's caching.

Software:

```
```text
```

```

User
↓
Application
↓
Cache
↓
Database

```

If data exists in cache:

```
```text
```

Cache HIT

```
If not:
```

```

```text
Cache MISS
↓
Database

```

Popular cache technologies include:

```
```text
```

Redis

Memcached

```

```

```
18. Why Do We Need Caching?
```

```
Suppose:
```

```

```text
Database request = 50 ms
Cache request = 2 ms

```

If the same information is requested millions of times, cache can dramatically reduce database load.

But caching introduces problems too:

- stale data
- cache invalidation
- memory usage
- consistency issues

This leads to an important principle:

Caching improves performance but increases system complexity.

```
---
```

19. Load Balancer

Suppose you have:

```
```text
```

1 server

and:

```
```text
1 million users
```

That server can become overloaded.

Instead:

```
```text
```

■■■■ Server 1

Users ■■■ Load ■■■■■■■■■■ Server 2

Balancer ■■■■ Server 3

■■■■ Server 4

A load balancer distributes incoming requests across multiple servers.

Think of it as a traffic controller.

---

# 20. Why Multiple Servers?

Because one server can fail.

Suppose:

```
```text
Server 1 → CRASH
```

If there is only one server:

```
```text
```

Application → DOWN

With multiple servers:

```
```text
Server 1 → CRASH

Server 2 → RUNNING
Server 3 → RUNNING
Server 4 → RUNNING
```

The service can continue operating.

This is the beginning of **fault tolerance**.

21. Horizontal vs Vertical Scaling

One of the most important concepts.

Vertical Scaling

Make one machine stronger.

```
```text
```

Before:

CPU: 4

RAM: 16 GB

↓

After:

CPU: 32

RAM: 128 GB

This is vertical scaling.

Also called:

> Scale up.

---

# 22. Horizontal Scaling

Add more machines.

```
```text
```

Server 1

Server 2

Server 3

Server 4

This is horizontal scaling.

Also called:

Scale out.

Modern distributed systems frequently rely heavily on horizontal scaling.

23. Why Horizontal Scaling Is Powerful

Imagine:

```
```text
```

1 server

gets 10,000 requests/sec.

You need 100,000 requests/sec.

You could theoretically make the machine 10× more powerful.

Or:

```
```text
10 servers × 10,000 requests/sec
```

Now the workload can be distributed.

But horizontal scaling creates new problems:

- shared state
- database bottlenecks
- session management
- load balancing
- distributed consistency

So scaling isn't simply:

"Add servers."

24. Stateless Server

A stateless server does not depend on local memory to remember important information between requests.

Example:

```
```text
```

Request 1 → Server A

Request 2 → Server B

Server B can still process Request 2 because necessary state is stored elsewhere.

For example:

```
```text
Application servers
    ↓
  Database
    ↓
    Redis
```

This makes horizontal scaling easier.

25. Stateful Server

A stateful server maintains important state locally.

Example:

```
```text
```

User session stored only on Server A

Then:

```
```text
Request 1 → Server A
Request 2 → Server B
```

Server B doesn't know the session.

This creates scaling problems.

26. Session

A session represents information associated with a user's interaction with a system.

For example:

```
```text
```

User ID

Authentication state

Preferences

Shopping cart

Sessions can be managed using:

- server-side storage
- cookies
- tokens
- Redis
- databases

---

# 27. Message Queue

Now imagine your application receives:

```
```text
1 million emails to send
```

Should the API wait until all emails are sent?

No.

Instead:

```
```text
```

User

↓

API

↓

Queue

↓

Worker

↓

Email service

The API puts a job into a queue.

Workers process it asynchronously.

---

# 28. Why Queues Are Useful

Queues provide:

### Decoupling

The API doesn't need to directly perform every operation.

### Asynchronous processing

Long-running work happens later.

### Load smoothing

If 100,000 jobs arrive suddenly:

```
```text
Queue
■■■ job
■■■ job
■■■ job
■■■ ...
```

Workers process them gradually.

Reliability

Jobs can sometimes be retried after failures.

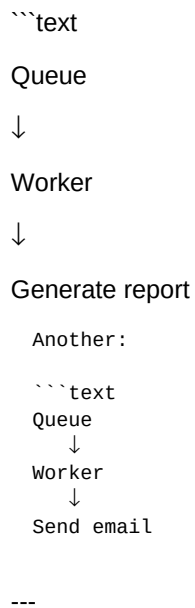
Examples:

- Kafka
- Google Pub/Sub
- RabbitMQ
- Amazon SQS

29. Worker

A worker is a process that performs background jobs.

For example:



30. Object Storage

Databases aren't always the best place to store large files.

For example:

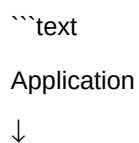
- PDFs
- images
- videos
- audio
- large datasets

Object storage is designed for this.

Examples:

- Google Cloud Storage
- Amazon S3
- Azure Blob Storage

Architecture:



Object Storage

↓

Large File

The database might store:

```
```text
document_id
file_name
storage_location
metadata
```

while the actual PDF lives in object storage.

This distinction becomes very important for your Project 1.

— — —

## 31. Microservices

Instead of building one enormous application:

```
```text
```

Everything

↓

One giant application

we can separate functionality:

```
```text
User Service
Document Service
Search Service
Test Service
Notification Service
```

Each service has a specific responsibility.

— — —

## 32. Monolith

A monolithic application packages most functionality together.

Example:

```
```text
```

Application

[REDACTED]

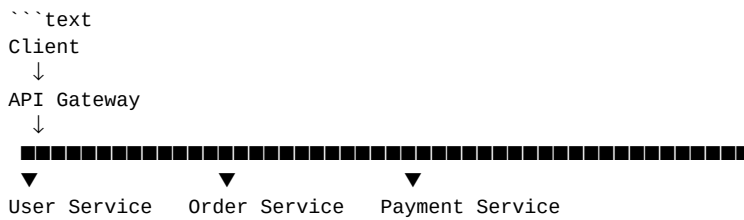
■ ■ ■

Users Orders Payments

This isn't inherently bad.

For smaller systems, a monolith can be simpler.

33. Microservices Architecture



Advantages can include:

- independent deployment
- independent scaling
- service isolation
- team ownership

Costs include:

- network communication
- distributed debugging
- deployment complexity
- data consistency challenges
- operational overhead

A critical system-design lesson:

More distributed does not automatically mean better.

34. Database Types

You will eventually need to understand several categories.

Relational

Examples:

```
``text
PostgreSQL
MySQL
Good for:
```

- structured data
- relationships
- transactions
- strong consistency requirements

Key-Value

Example:

```
```text
Redis
```

Concept:

```
```text
```

key → value

Example:

```
```text
user:123 → session information
```

---

## Document Database

Examples:

```
```text
```

MongoDB

Data is represented as documents.

Wide-Column

Examples:

```
```text
Cassandra
Bigtable
```

Useful for certain large-scale distributed workloads.

---

## Graph Database

Designed around:

```
```text
```

nodes

relationships

Useful for some relationship-heavy problems.

35. CAP Theorem

This is a major system-design topic.

CAP stands for:

```
```text
C = Consistency
A = Availability
P = Partition tolerance
```

The basic idea is that in a distributed system experiencing a network partition, you cannot simultaneously guarantee both perfect consistency and availability.

Let's understand the words.

---

## 36. Consistency

After data is updated, clients see the appropriate/current value according to the system's consistency model.

Example:

```
```text
```

Account balance = ■1000

User withdraws ■500.

The system should not incorrectly show:

```
```text
■1000
```

to a reader when strong consistency is required.

---

## 37. Availability

The system continues responding to requests.

For example:

```
```text
```

Server A fails

but:

```
```text
Server B
Server C
```

continue serving users.

---

## 38. Partition

A network partition occurs when parts of a distributed system cannot communicate reliably.

Imagine:

```text

Data Center A

X

network failure

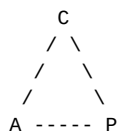
Data Center B

Both sides may still be running but cannot communicate correctly.

39. CAP Mental Model

Think:

```text



The important interview statement is:

*When a network partition occurs, a distributed system must choose between stronger consistency and continued availability for affected operations.*

Don't memorize CAP as "pick any two" without context. The partition scenario is the key.

---

## 40. Latency

Latency means:

*How long does an operation take?*

For example:

```text

Request

↓

100 ms

↓

Response

Latency = 100 ms.

41. Throughput

Throughput means:

> How much work can the system handle in a given amount of time?

Example:

```
```text
10,000 requests / second
```

That's throughput.

---

## 42. Latency vs Throughput

Imagine a checkout counter.

### Latency

How long does one customer wait?

### Throughput

How many customers can the store serve per minute?

A system can sometimes have:

```
```text
```

low latency

but limited throughput, or high throughput with higher individual latency.

You need to consider both.

43. Availability

Availability measures how often the system is operational.

For example:

```
```text
99%
99.9%
99.99%
99.999%
```

The number of 9s matters enormously.

---

## 44. Reliability

Reliability is about the system performing correctly over time.

A system could technically be available but produce incorrect results.

Therefore:

```text

Availability $\neq$ Reliability

A system should ideally be:

```text  
Available  
+  
Correct  
+  
Reliable

---

## 45. Fault Tolerance

Fault tolerance means the system continues functioning despite failures.

Example:

```text

Database replica 1 → FAILED

Database replica 2 → ACTIVE

The system continues operating.

46. Redundancy

Redundancy means having additional components so that one failure doesn't destroy the entire system.

Example:

```text  
Server A  
Server B  
Server C

instead of:

```text

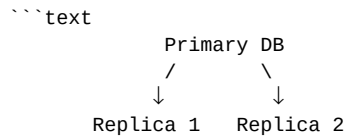
Server A

Redundancy generally increases resilience but also increases cost.

47. Replication

Replication means keeping multiple copies of data.

For example:



If the primary fails, another node may potentially take over depending on the database architecture.

Replication can improve:

- availability
- read scalability
- disaster recovery

But introduces:

- replication lag
- consistency complexity
- operational complexity

48. Sharding

Suppose one database contains:

```
``text
```

1 billion users

You can partition the data across multiple database servers.

For example:

```
``text
Users A-F → DB1
Users G-M → DB2
Users N-S → DB3
Users T-Z → DB4
```

This is a simple conceptual example of sharding.

Sharding is:

Splitting data across multiple machines.

49. Partitioning vs Sharding

These words are related but not always identical.

A useful beginner mental model:

Partitioning

Divide data logically into pieces.

Sharding

Distribute those pieces across multiple machines/nodes.

50. Database Bottleneck

Suppose:

```
```text
```

100 application servers

↓

one database

The application may scale horizontally while the database becomes the bottleneck.

This is a critical system-design insight:

> **Scaling one layer does not automatically scale the whole system.**

You must identify bottlenecks across the entire architecture.

---

# 51. Bottleneck

A bottleneck is the part of the system limiting overall performance.

Example:

```
```text
```

Application → 100 servers

Database → 1 server

The database might become the bottleneck.

Another example:

```
```text
```

↓

The worker could be the bottleneck.

— — —

Instead of every user downloading it from one central server:

```
```text
```

Europe → Europe edge

...

```
```text
google.com
```

Think of DNS as the internet's directory service.

---

```
```text
```

↓

Reverse Proxy

↓

Backend

It can provide:

- routing
- TLS termination
- load balancing
- caching
- security controls

55. API Gateway

An API Gateway can act as the front door to multiple backend services.

```text

```

 ■■■■ User Service
Client → Gateway ■■■■■■ Order Service
 ■■■■ Search Service
 ■■■■ Payment Service
```

It can handle things such as:

- authentication
- routing
- rate limiting
- request transformation
- logging

---

## 56. Rate Limiting

Imagine someone sends:

```text

1 million requests/second

to your API.

That could overwhelm the service.

Rate limiting says:

> A client may make only a certain number of requests during a time window.

Example:

```text

100 requests/minute

Architecture:

```text

Client

↓

Rate Limiter

↓

API

57. Why Rate Limiting Matters

It helps protect against:

- accidental overload
- abusive clients
- denial-of-service-like traffic
- runaway applications

Common strategies include:

- token bucket
- leaky bucket
- fixed window
- sliding window

58. Authentication

Authentication answers:

> "Who are you?"

Example:

```
```text
Username + Password
```

or:

```
```text
```

Token

59. Authorization

Authorization answers:

> "What are you allowed to do?"

Example:

```
```text
Admin → delete documents
User → read documents
```

Remember:

```
```text
```

Authentication = identity

Authorization = permissions

60. Observability

A production system needs to tell engineers what is happening.

Observability generally includes:

```
```text
Logs
Metrics
Traces
```

---

## 61. Logs

Logs are records of events.

Example:

```
```text
2026-09-16
POST /documents
document_id=123
status=201
```

Logs answer:

> What happened?

62. Metrics

Metrics are numerical measurements.

Examples:

```
```text
Requests/sec
CPU usage
Memory usage
Error rate
Latency
Queue depth
```

Metrics answer:

*How much? How often? How fast?*

---

## 63. Traces

A distributed request might travel through:

```
```text
```

API

↓

Document Service

↓

Search Service

↓

Database

A trace lets you follow that request across components.

It answers:

> Where did the request spend its time?

This will become especially important for your AI Engineering platform.

64. Monitoring

Monitoring watches system health.

For example:

```
```text
CPU > 90%
```

could trigger an alert.

Or:

```
```text
```

Error rate > 5%

could trigger an alert.

65. Scalability

Scalability means the system can handle increasing workload by adding resources or changing architecture appropriately.

Workload can increase through:

```
```text
more users
more requests
more data
larger files
more ML inference
more concurrent jobs
```

---

## 66. A Very Important Distinction

### Scalability

Can the system handle more workload?

## Performance

How efficiently does it handle the workload?

## Reliability

Does it continue functioning correctly?

## Availability

Can users access it?

## Durability

Will data survive failures?

These are related but different concepts.

---

## 67. Durability

Durability means that once data is successfully stored, it should not disappear unexpectedly.

For example:

```text

Payment completed

You don't want the payment record disappearing because one server crashed.

Databases and storage systems provide various durability guarantees.

68. Consistency

Consistency describes what different users/servers observe regarding data.

There are many consistency models.

At a beginner level, understand:

```text

Strong consistency

versus

```text

Eventual consistency

69. Strong Consistency

After a successful write, subsequent reads see the updated value according to the system's guarantee.

Example:

```
```text
Write:
balance = █500

Immediately read:
balance = █500
```

This is useful when correctness requires immediate visibility.

---

## 70. Eventual Consistency

Different replicas may temporarily have different values.

Eventually:

```
```text
Replica A → █500
Replica B → █500
Replica C → █500
```

This can improve availability/scalability in some architectures but means users may temporarily see stale data.

71. Distributed Systems

Now we can define one of the most important terms.

A distributed system is:

> A collection of independent computers that communicate over a network and work together as one logical system.

For example:

```
```text
Server A
Server B
Server C
Database A
Database B
Queue
Cache
```

All work together.

---

## 72. Why Distributed Systems Are Difficult

Because networks fail.

Computers fail.

Processes crash.

Messages are delayed.

Messages can be duplicated.

Data can become temporarily inconsistent.

Servers can become overloaded.

Therefore:

*Distributed systems are fundamentally about dealing with partial failure and coordination.*

This is a major idea to remember for Google system design interviews.

---

## 73. Synchronous Communication

Example:

```text

Service A ■■■■■request■■■■■ Service B

■■■■■response■■■■■

A waits for B.

74. Asynchronous Communication

Example:

```text

Service A

↓

Queue

↓

Service B

A doesn't necessarily wait for B to finish.

This is useful for background processing.

---

## 75. Retry

Suppose:

```
```text
```

Service A → Service B

and B temporarily fails.

A may retry.

But retries can cause problems.

Imagine a payment request:

```
```text
Charge ■100
```

Request reaches payment service.

Payment succeeds.

Response is lost.

Client retries.

Now you might accidentally charge twice.

This leads to a critical system-design concept.

---

## 76. Idempotency

An operation is idempotent if repeating it produces the same intended final effect as performing it once.

For example:

```
```text
```

PUT /users/123

setting:

```
```text
name = "John"
```

multiple times can result in the same state.

Payment operations require special care.

You may use an:

```
```text
```

idempotency key

so the server recognizes duplicate requests.

77. Timeout

Never assume another service will respond immediately.

Example:

```
```text
Service A
 ↓
Service B
```

Service B could hang.

Therefore A should use a timeout:

```
```text
```

Wait 2 seconds

↓

No response

↓

Fail/Retry/Fallback

Timeouts prevent resources from being held indefinitely.

78. Circuit Breaker

Imagine Service B is completely broken.

Without protection:

```
```text
A → B → failure
A → B → failure
A → B → failure
A → B → failure
...

```

A circuit breaker can detect repeated failures and temporarily stop sending requests.

Conceptually:

```
```text
```

CLOSED

↓

many failures

↓

OPEN

↓

stop requests

↓

wait

↓

HALF-OPEN

↓

test

This prevents cascading failures.

79. Cascading Failure

Suppose:

```text

A → B → C → D

D becomes slow.

Then:

```text

C waits for D

B waits for C

A waits for B

Soon the entire system may become overloaded.

This is a cascading failure.

System design must consider:

- timeouts
- retries
- circuit breakers
- backpressure
- load shedding

80. Backpressure

Suppose:

```text

Producer → 100,000 jobs/sec

Worker → 10,000 jobs/sec

The producer is faster than the consumer.

The queue grows:

```text

10k

50k

100k

500k

1M

The system needs a strategy.

Backpressure means slowing/restricting producers or otherwise controlling flow when downstream capacity is insufficient

81. System Design Starts With Requirements

This is the **first step in every system-design interview.**

Never immediately draw:

```text

Load Balancer

Redis

Kafka

PostgreSQL

Kubernetes

First ask:

*What are we building?*

---

## 82. Functional Requirements

Functional requirements describe:

*What the system must do.*

Example for a URL shortener:

```text

User can create short URL

User can access short URL

System redirects to original URL

83. Non-Functional Requirements

These describe:

> How the system should behave.

Examples:

```text

Low latency

High availability

Scalability

Security

Durability

Reliability

---

## 84. Functional vs Non-Functional

Example:

### Functional

```text

Create a document.

Non-functional

```text

Document creation should remain available  
during server failures and support high traffic.

---

## 85. Clarifying Questions

In an interview, clarify things such as:

### Users

```text

How many users?

Traffic

```text

How many requests per second?

### Data

```text

How much data?

Read/write ratio

```text

Are reads much more common than writes?

### Latency

```text

What latency is acceptable?

Availability

```text

How much downtime is acceptable?

### Consistency

```
```text
```

Do reads need to immediately reflect writes?

These questions determine architecture.

...

86. Scale Estimation

This is where system design becomes quantitative.

Suppose:

```
```text
10 million users
```

and:

```
```text
```

10% active daily

Then:

```
```text
1 million daily active users
```

If each makes:

```
```text
```

20 requests/day

then:

```
```text
20 million requests/day
```

Now convert to average requests per second:

```
```text
```

20,000,000

■■■■■■■■■■

86,400

 $\approx$

```
```text
231 requests/sec
```

Then account for peak traffic.

Perhaps peak is several times the average.

This is why requirements matter.

— — —

## 87. Why We Estimate Traffic

Because architecture depends on scale.

For:

```
```text
```

100 requests/day

you don't need the same architecture as:

```
```text
```

100 million requests/sec

---

## 88. Storage Estimation

Suppose:

```
```text
```

1 million users

Each generates:

```
```text
```

10 KB/day

Then:

```
```text
```

$1,000,000 \times 10 \text{ KB}$

$\approx$

```
```text
```

10 GB/day

Over a year:

```
```text
```

$\approx 3.65 \text{ TB/year}$

Now you can start asking:

- Can one database handle this?
- Do we need object storage?
- Do we need partitioning?
- Do we need archival?

89. API Design

After requirements and scale, define APIs.

Example:

```
```text
```

POST /urls

GET /urls/{id}

For a document system:

```
```text
```

POST /documents

GET /documents

GET /documents/{id}

PUT /documents/{id}

DELETE /documents/{id}

This directly connects to your Project 1.

90. Data Model

Next ask:

> What information must we store?

For Project 1:

```
```text
```

Document

-----

id

name

type

version

status

timestamps

Later you'll add concepts such as:

```
```text
```

chunks

embeddings

requirements

tests

test_results

releases

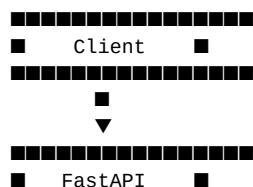
This eventually becomes a much richer system.

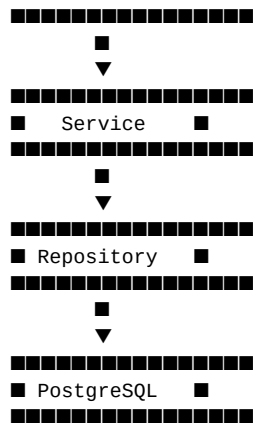
91. Architecture

Only now should you draw the major components.

For a simple document service:

```
```text
```





That is already a system design.

---

## 92. Then Ask: Where Is the Bottleneck?

For example:

```
```text
```

100 API servers

↓

1 PostgreSQL database

Database may become bottleneck.

Possible solutions:

```
```text
```

Indexes

Caching

Read replicas

Partitioning

Sharding

But don't blindly add all of them.

---

## 93. Trade-Offs

System design is largely about **trade-offs**.

There is rarely one perfect architecture.

Example:

### Cache

Pros:

```text

faster reads

lower DB load

Cons:

```text  
stale data  
invalidation complexity  
extra infrastructure

Therefore:

*Use caching when its benefits justify the complexity.*

---

## 94. Cost

Every component costs money and operational effort.

Imagine:

```text

100 servers

10 databases

Redis cluster

Kafka cluster

Kubernetes

multiple regions

That may be unnecessarily complex for a small application.

Good system design considers:

```text  
Performance  
+  
Reliability  
+  
Scalability  
+  
Cost  
+  
Complexity

---

## 95. Security

Every system-design answer should eventually consider:

## **Authentication**

Who is the user?

## **Authorization**

What can they do?

## **Encryption**

Protect data in transit and at rest.

## **Secrets**

Don't store passwords/API keys in source code.

## **Input validation**

Don't trust client input.

## **Rate limiting**

Protect APIs from abuse.

## **Audit logs**

Record important security-sensitive operations.

---

## **96. Disaster Recovery**

Imagine an entire data center fails.

What happens?

You need to think about:

```text

Backups

Replication

Multiple regions

Recovery procedures

RTO

RPO

97. RTO

Recovery Time Objective:

> How quickly must the system recover after a failure?

Example:

```
```text
RTO = 30 minutes
```

---

## 98. RPO

Recovery Point Objective:

*How much data loss can the business tolerate?*

Example:

```
```text
```

RPO = 5 minutes

Meaning, conceptually, losing up to about five minutes of recent data may be acceptable under the requirement.

99. Single Point of Failure

A component is a single point of failure when its failure can bring down the system.

Example:

```
```text
 Server
 ■
 ▼
 One Database
```

If that database fails:

```
```text
```

System → unavailable

You should identify these explicitly during system design.

100. Health Checks

Services should expose health information.

For example:

```
```text
GET /health
```

Your Project 1 already has a health endpoint.

A health endpoint can help infrastructure determine whether a service is functioning.

But health checks themselves need thoughtful design; simply returning ``OK`` doesn't necessarily prove dependencies such as databases are healthy.

---

## 101. Deployment

Eventually software needs to run somewhere.

Typical progression:

```
```text
```

Local machine

↓

Docker

↓

Kubernetes

↓

Cloud

Your roadmap follows exactly this progression.

102. Docker

Docker packages an application and its dependencies into a container image.

Conceptually:

```
```text
```

Application

+

Dependencies

+

Runtime

↓

Container

This makes environments more reproducible.

---

## 103. Kubernetes

Kubernetes orchestrates containers.

It can manage:

- deployment
- scaling
- service discovery
- restarting failed containers
- rolling updates

You will study Kubernetes later in your roadmap, not on your current database-focused day.

---

## 104. Cloud

Cloud platforms provide infrastructure/services on demand.

For your roadmap:

```text

GCP

will eventually provide services for:

- compute
- databases
- storage
- networking
- Kubernetes
- monitoring
- messaging

105. The Most Important System Design Framework

For your Google preparation, use this structure every time:

```text

1. Requirements
- ↓
2. Scale estimation
- ↓
3. APIs
- ↓
4. Data model
- ↓
5. High-level architecture
- ↓
6. Detailed component design
- ↓
7. Bottlenecks
- ↓
8. Caching
- ↓
9. Database scaling
- ↓
10. Consistency
- ↓
11. Reliability
- ↓
12. Security
- ↓
13. Observability

- ↓  
14. Cost  
↓  
15. Failure scenarios  
↓  
16. 10× / 100× scaling

Memorize this **process**, not individual architectures.

---

## 106. Example: Design a URL Shortener

Let's see the framework in action.

Requirement:

*Build a service like Bitly.*

User gives:

```text

`https://example.com/very/long/url`

System returns:

```text

`https://short.com/aB7x`

---

## 107. Functional Requirements

```text

Create short URL

Redirect short URL

Potentially later:

```text

Analytics

Expiration

Custom aliases

But don't automatically include everything.

---

## 108. APIs

```text

POST /urls

Input:

```
```json
{
 "url": "https://example.com"
}
```

Response:

```
```json
{
  "short_url": "https://short.com/aB7x"
}
```

Redirect:

```
```text
GET /aB7x
```

---

## 109. Data Model

```
```text
```

urls

short_code

long_url

created_at

expires_at

110. Basic Architecture

```
```text
```

Client

↓

Load Balancer

↓

Application Servers

↓

Cache

↓

Database

For redirects:

```
```text
```

GET /aB7x

↓

Application

↓

↓

↓

...

Reads = 100x writes

```
```text
```

**XXXXXXXXXXXXXXXXXXXX**

**■■■■■▲■■■■■■■■■■**



▼

\*\*\*

↓

Read database

---

## Page 51

For:

```text

Application

→ use multiple instances.

For:

```text  
Cache

→ fall back to database when appropriate.

For:

```text

Database

→ replication/failover/backups depending on requirements.

For:

```text  
Queue

→ durable messaging and retry strategy.

For:

```text

Network

→ timeout/retry/circuit-breaker strategy.

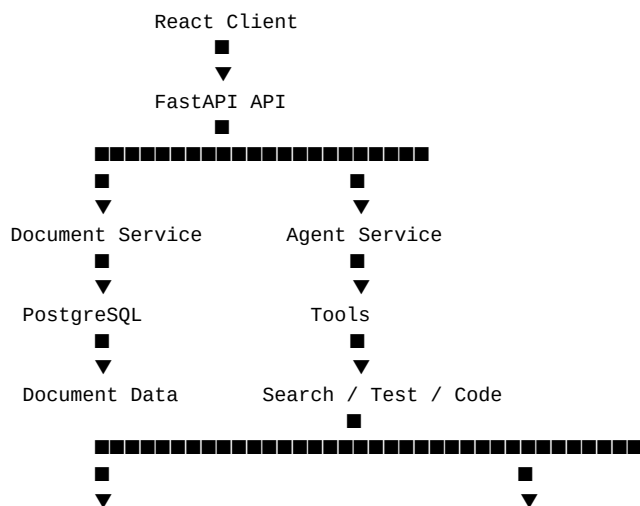
This mindset is extremely important.

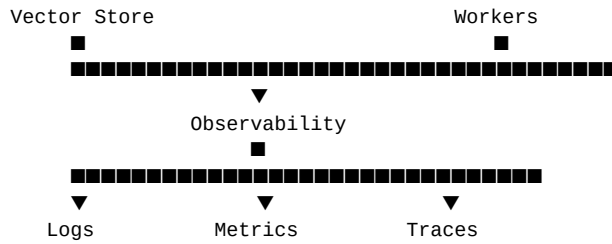
114. Your Project 1 Through a System Design Lens

Your AI Engineering Change & Test Intelligence Platform is eventually going to become a very good system-design project

The conceptual architecture is:

```text





But **do not build all of this now**.

Your roadmap deliberately builds it incrementally.

---

## 115. How Your Project Evolves

You are essentially learning system design by building the system.

### Phase 1

```
```text
```

FastAPI

Learn:

```
```text
```

API

Request

Response

Validation

### Phase 2

```
```text
```

Service Layer

Learn:

```
```text
```

business logic

separation of concerns

### Phase 3

```
```text
```

Repository

Learn:

```
```text
```

persistence abstraction

### Phase 4

```text

PostgreSQL

Learn:

```text  
durable storage  
transactions  
database design

## Phase 5

```text

Redis

Learn:

```text  
caching

## Phase 6

```text

Queues/workers

Learn:

```text  
asynchronous processing

## Phase 7

```text

RAG

Learn:

```text  
retrieval architecture  
vector search

## Phase 8

```text

Agents

Learn:

```text  
orchestration  
tool calling

## Phase 9

```text

Docker/Kubernetes/GCP

Learn:

```
```text
deployment
scaling
cloud infrastructure
```

## Phase 10

```
```text
```

Observability

Learn:

```
```text
production operations
```

That is **system design through implementation**.

---

## 116. System Design Interview Mindset

When an interviewer says:

*"Design YouTube."*

Don't panic.

Don't immediately draw:

```
```text
```

Kafka

Redis

Kubernetes

Bigtable

Instead:

Step 1

Clarify:

> What part of YouTube?

For example:

```
```text
video upload
video streaming
recommendations
comments
creator support
```

## Step 2

Define requirements.

### **Step 3**

Estimate scale.

### **Step 4**

Define APIs.

### **Step 5**

Define data.

### **Step 6**

Draw simple architecture.

### **Step 7**

Identify bottlenecks.

### **Step 8**

Scale the bottlenecks.

### **Step 9**

Discuss failures.

### **Step 10**

Discuss trade-offs.

This demonstrates engineering reasoning.

---

## **117. The Most Important Questions to Ask About Any System**

Whenever you see a system, ask:

### **Users**

*Who uses it?*

## **Function**

*What does it do?*

## **Traffic**

*How many requests?*

## **Data**

*What data exists?*

## **Storage**

*Where does data live?*

## **Communication**

*How do components communicate?*

## **Performance**

*What is the latency requirement?*

## **Scale**

*What happens at 10× traffic?*

## **Failure**

*What happens if a component crashes?*

## **Consistency**

*What happens if replicas disagree?*

## **Security**

*Who can access what?*

## **Observability**

*How do engineers know something is broken?*

## **Cost**

*What does this architecture cost?*

These questions form the foundation of system design.

---

## 118. The Four Levels You Should Eventually Master

For your Google preparation, think of system design as four layers.

### Level 1 — Components

Know:

```text

API

Server

Database

Cache

Queue

Load Balancer

Storage

Level 2 — Distributed Systems

Understand:

```text

Replication

Partitioning

Sharding

Consistency

Availability

Fault tolerance

Retries

Timeouts

Idempotency

---

### Level 3 — Production Systems

Understand:

```text

Monitoring

Logging

Tracing

Security

CI/CD

Deployment

Autoscaling

Disaster recovery

Cost

Level 4 – Architecture Reasoning

Be able to explain:

> Why did you choose this architecture instead of another?

That final level is what separates memorization from genuine system-design ability.

119. Your System Design Study Progression

Based on your Google Career roadmap, don't try to master everything immediately.

Your progression should be:

```
```text
Current
↓
Backend fundamentals
↓
PostgreSQL
↓
API architecture
↓
Caching
↓
Queues
↓
Distributed systems
↓
System design fundamentals
↓
RAG architecture
↓
LLM inference
↓
Agent architecture
↓
ML serving
↓
Kubernetes
↓
GCP
↓
Observability
↓
Production system design
```
```

120. System Design Cheat Sheet

Keep this mental model:

```
```text
```

## SYSTEM DESIGN



## Requirements Scale Data



## APIs Traffic/Storage Database



## Architecture



## Cache Queue Storage



## Scalability



## Replication Sharding Partitioning



## Reliability



## Timeout Retry Failover



## Security



## Observability



## Logs Metrics Traces



## Cost

---

### # 121. What You Should Be Able to Explain After These Notes

Before moving deeper, you should be able to explain in your own words:

- What system design is
- Why system design is needed
- Client vs server
- API
- HTTP
- Database
- Cache
- Load balancer
- Horizontal scaling
- Vertical scaling
- Stateless vs stateful
- Message queue
- Worker
- Object storage
- Monolith
- Microservices
- Replication
- Sharding
- Partitioning
- CAP theorem
- Consistency
- Availability
- Reliability
- Durability
- Fault tolerance
- Latency
- Throughput
- Rate limiting
- Authentication
- Authorization
- Logs
- Metrics
- Traces
- CDN
- DNS
- API gateway
- Timeouts
- Retries
- Idempotency
- Circuit breakers
- Cascading failures
- Backpressure
- RTO
- RPO
- Bottlenecks
- Trade-offs
- Requirements
- Scale estimation
- API design
- Data modeling
- Architecture design

And most importantly:

> **\*\*You should be able to explain why a component is needed, what problem it solves, what new problems it introduces, a**

That is the foundation we will build on for your **\*\*Google system-design preparation\*\***.